

Práctica 1. Del programa fuente a los tokens

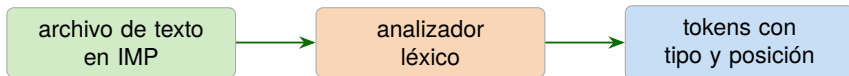
Construcción guiada del analizador léxico para IMP

Manuel Soto Romero Araceli Liliana Reyes Cabello

Colegio de Ciencia y Tecnología, UACM

Semestre 2026-2

Reto de la práctica



Pregunta guía

¿Qué debemos especificar, construir y probar para que la transformación sea reproducible?

Producto que construiremos

Código

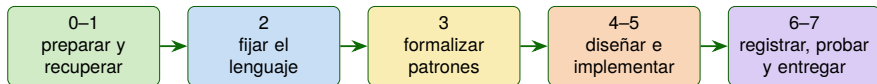
- ▶ AFD de localidades.
- ▶ Reglas léxicas con LARK.
- ▶ Tabla inicial de localidades.
- ▶ Controlador y pruebas.

Evidencias

- ▶ Entradas correctas y errores.
- ▶ Resultados esperados y reales.
- ▶ README.md y dependencias.
- ▶ Historial de GIT y reporte.

El lexer quedará preparado para alimentar al analizador sintáctico.

Ruta de trabajo



Ritmo

Predicción → cambio pequeño → prueba → comparación → evidencia.

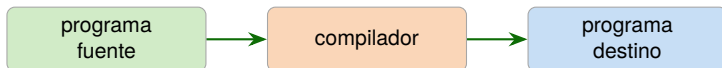
El compilador como software de sistemas

Conecta representaciones

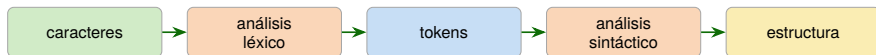
Recibe un programa descrito en un lenguaje y produce otra representación que conserva el comportamiento definido.

Organiza responsabilidades

Cada fase recibe información, realiza una tarea delimitada y entrega un resultado a la fase siguiente.



Abrimos el compilador



Foco de la práctica

Sólo construiremos el tramo caracteres → tokens. No comprobaremos todavía la estructura completa, las declaraciones ni los tipos.

Contrato del analizador léxico

Entrada	flujo de caracteres del programa fuente
Tarea	agrupar caracteres, reconocer lexemas, clasificarlos y conservar atributos
Salida	secuencia de tokens con información de sus apariciones
Consumidor	analizador sintáctico

Límite

Reconocer x como localidad no permite decidir si fue declarada, qué tipo tiene o qué valor almacena.

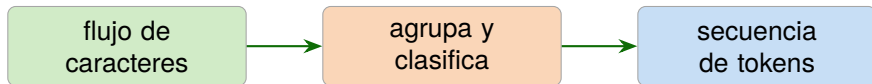
Trabajo con la guía práctica

Guía de la Práctica 1

Pasos 0 y 1

- ▶ Ejecutar la versión mínima antes de modificarla.
- ▶ Completar el recorrido y el contrato léxico.
- ▶ Comparar el límite de responsabilidad en parejas.

Función del analizador léxico



- ▶ Puede omitir espacios cuando la convención lo permite.
- ▶ Conserva información necesaria para fases posteriores.
- ▶ Informa dónde comienza una unidad o un error.

Token, lexema, patrón y atributo

Lexema	caracteres concretos encontrados, por ejemplo x2
Patrón	forma admitida para un conjunto de lexemas
Token	nombre de categoría que comunica el reconocimiento
Atributo	información de la aparición: texto, valor o posición

Cuatro preguntas

¿Qué apareció?, ¿qué forma satisface?, ¿qué nombre se emite? y ¿qué información debe conservarse?

IMP: conjuntos sintácticos

números
N

valores
T

localidades
Loc

expresiones
Aexp

expresiones
Bexp

comandos
Com

Fuente de trabajo

Winskel presenta IMP como un lenguaje imperativo pequeño con expresiones aritméticas, expresiones booleanas y comandos que modifican un estado.

Sintaxis abstracta de IMP

$$a ::= n \mid X \mid a_0 + a_1 \mid a_0 - a_1 \mid a_0 \times a_1,$$
$$b ::= \text{true} \mid \text{false} \mid a_0 = a_1 \mid a_0 \leq a_1 \mid \neg b \mid b_0 \wedge b_1 \mid b_0 \vee b_1,$$
$$c ::= \text{skip} \mid X := a \mid c_0; c_1 \mid \text{if } b \text{ then } c_0 \text{ else } c_1 \mid \text{while } b \text{ do } c.$$

Qué obtenemos

Un vocabulario que debe poder escribirse y reconocerse. La sintaxis abstracta no fija por sí sola todas las precedencias ni todos los detalles del texto fuente.

Abstracta no significa concreta

Sintaxis abstracta

Indica cómo se construyen expresiones y comandos. Ayuda a pensar en su estructura.

Escritura concreta

Decide qué caracteres se usan, cómo se agrupa y qué convenciones deberá reconocer el lexer.

Decisión visible

La práctica documentará una convención ASCII. No presentaremos esa adaptación como si fuera una regla literal de Winskel.

Convención concreta del curso

Abstracta	Archivo	Token
$\times$	$*$	TIMES
$\leq$	$\leq$	LE
$\neg, \wedge, \vee$	!, &&,	NOT, AND, OR
$:=, ;$	$:=, ;$	ASSIGN, SEMI
agrupación	(,)	LPAR, RPAR

Numerales

NUM contiene dígitos; - se emite aparte como MINUS. Así la fase léxica no cambia de decisión según el contexto.

Inventario léxico

Datos	LOC, NUM
Valores	TRUE, FALSE
Palabras	SKIP, IF, THEN, ELSE, WHILE, DO
Aritmética	PLUS, MINUS, TIMES
Booleanos	EQ, LE, NOT, AND, OR
Control	ASSIGN, SEMI, LPAR, RPAR
Omitido	espacios, tabuladores y saltos de línea

Los nombres formarán parte de la interfaz de la práctica siguiente.

Guía de la Práctica 1

Paso 2

- ▶ Leer la sintaxis abstracta de IMP.
- ▶ Completar atributos del inventario.
- ▶ Justificar palabras reservadas, signo y contrato futuro.

Del patrón a una especificación



Dos problemas distintos

La expresión regular **especifica** los lexemas; el autómata **reconoce** si una cadena satisface esa forma.

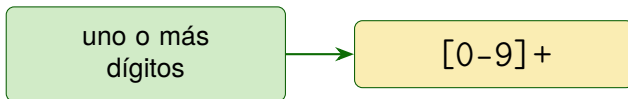
Notación que necesitaremos

Notación	Función
rs	concatenar una forma seguida de otra
$r s$	elegir entre dos formas
r^*	repetir cero o más veces
r^+	repetir una o más veces
$r^?$	permitir cero o una aparición
$[A-Za-z]$	elegir un carácter de una clase

Lectura antes de código

Cada símbolo debe relacionarse con ejemplos que acepta y contraejemplos que rechaza.

Modelo: numerales



Acepta	Rechaza como un solo NUM
0, 25, 2026	cadena vacía, X, -3

El signo menos será otro token.

Ahora: localidades

Descripción

Una o más letras seguidas opcionalmente por cero o más dígitos.

LOC = ?

Candidatos	Contraejemplos
X, contador, X2, Total25	2X, X_2, X2Y

Antes de proponer

La expresión debe aceptar todos los candidatos y rechazar todos los contraejemplos.

Trabajo con la guía práctica

Guía de la Práctica 1

Pasos 3.1 y 3.2

- ▶ Leer la expresión de los numerales.
- ▶ Derivar el patrón de localidades.
- ▶ Probar candidatos y contraejemplos antes del AFD.

Autómata finito determinista

Componentes

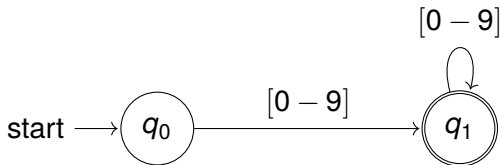
- ▶ estados;
- ▶ alfabeto;
- ▶ transiciones;
- ▶ estado inicial;
- ▶ estados finales.

Funcionamiento

Consume un símbolo a la vez.
Una cadena se acepta cuando se agota la entrada en un estado final.

Determinista: para cada par estado–símbolo hay como máximo una transición.

Modelo: AFD para $[0-9]^+$



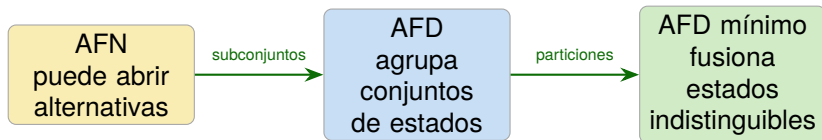
q_0

Todavía no se ha leído ningún dígito. No es final.

q_1

Ya se leyó al menos un dígito. Es final y puede consumir más.

De AFN a AFD y a una versión mínima



- ▶ El cierre ε reúne estados alcanzables sin consumir entrada.
- ▶ Cada conjunto alcanzable se convierte en un estado determinista.
- ▶ Se eliminan estados inalcanzables.
- ▶ Sólo se fusionan estados que no pueden distinguirse con entradas futuras.

Qué debe recordar el AFD de localidades



Estados finales

q_L y q_D aceptan. No pueden fusionarse: desde q_L aún se admite otra letra; desde q_D , no.

Tarea

Completar todas las transiciones y comprobarlas con candidatos y contraejemplos.

Trabajo con la guía práctica

Guía + scripts

Paso 3.3 y `afd_localidades.py`

- ▶ Completar la tabla del AFD.
- ▶ Implementar las transiciones faltantes.
- ▶ Habilitar las pruebas del paso 3.3.

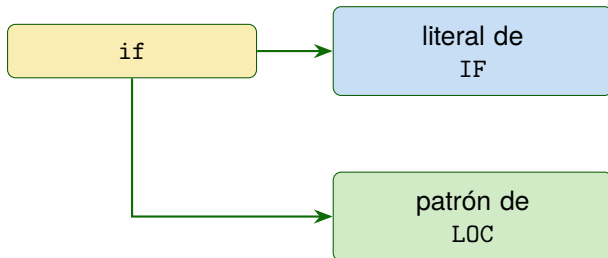
Diseñar antes de programar

Token	Patrón	Acción	Atributo
LOC	por completar	emitir y registrar	lexema y posición
NUM	[0-9] +	emitir	lexema o valor
IF	if "	emitir	no requiere
WS	espacios	omitir	posición actualizada

Contrato ejecutable

La tabla debe coincidir con la gramática léxica, la salida observada y los resultados esperados de las pruebas.

Una colisión: if también parece LOC



```
IF.2: /if(?![A-Za-z0-9])/
LOC: /[A-Za-z]+[0-9]*/
```

Decisión

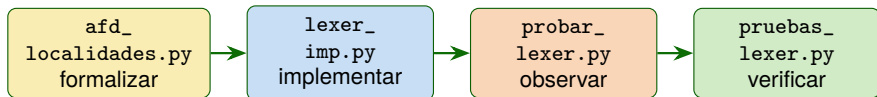
La prioridad resuelve el empate exacto; el límite evita cortar una localidad más larga. Por eso `if` es IF, pero `if2` e `ifX` son LOC.

Guía de la Práctica 1

Paso 4

- ▶ Completar la tabla de reglas.
- ▶ Predecir `if`, `if2` e `ifX`.
- ▶ Escribir la secuencia esperada antes de modificar el lexer.

Los scripts siguen la misma ruta



Punto de partida

La versión inicial sólo reconoce LOC y omite espacios. Los TODO indican los incrementos; las pruebas se habilitan por paso.

Lexer mínimo con LARK

```
from lark import Lark

GRAMATICA_LEXICA = r"""
LOC: /[A-Za-z]+[0-9]*/
%import common.WS
%ignore WS
"""

lexer = Lark(
    GRAMATICA_LEXICA,
    parser=None,
    lexer="basic",
)

tokens = list(lexer.lex("X"))
```

Separación de fases

`parser=None` y `lex()` permiten observar el lexer sin introducir todavía una gramática sintáctica.

Trabajo con la guía práctica

Guía + scripts

Paso 5.2

- ▶ Incorporar NUM.
- ▶ Habilitar la prueba de localidades y numerales.
- ▶ Registrar el cambio y su resultado.

Extender sin perder el control



- ▶ Una nueva categoría se acompaña de su prueba.
- ▶ Las palabras reservadas se verifican contra LOC.
- ▶ Los operadores de dos caracteres se prueban completos.
- ▶ La salida esperada no se ajusta para ocultar un error.

Trabajo con la guía práctica

Guía + scripts

Pasos 5.3 y 5.4

- ▶ Incorporar palabras reservadas con prioridad y límite léxico.
- ▶ Incorporar operadores y delimitadores.
- ▶ Ejecutar `flujo.imp` y `control.imp`.

Atributos observables

Tipo	Lexema	Línea	Columna
LOC	X	1	1
ASSIGN	:=	1	3
NUM	2	1	6

Mismo nombre, distinta aparición

Dos tokens LOC comparten categoría, pero pueden conservar lexemas y posiciones diferentes.

No adelantar semántica

La fase léxica no asignará tipos ni decidirá si la localidad fue declarada.

Tabla inicial de localidades

Lexema	Primera posición	Apariciones
X	1 : 1	2
Y	1 : 10	1

Propósito limitado

Registrar identidades y ubicaciones útiles. Los tipos, valores y alcances se incorporarán cuando exista información semántica.

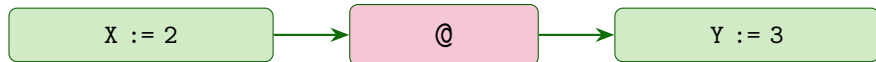
Trabajo con la guía práctica

Guía + scripts

Paso 6

- ▶ Observar tipo, lexema, línea y columna.
- ▶ Completar `construir_tabla_localidades`.
- ▶ Habilitar la prueba de la tabla.

Cuando ningún patrón puede avanzar



Error léxico

@ no inicia ninguna categoría aprobada. Informaremos línea y columna y detendremos esta ejecución; no inventaremos una corrección.

Matriz mínima de pruebas

Clase	Ejemplo	Qué comprueba
Mínimo correcto	X	versión inicial
Varias categorías	X := 25;	segmentación y atributos
Colisión	if if2	prioridad y límite léxico
Operador compuesto	<=, :=	unidad completa
Espacios y líneas	archivo multilinea	omisión y posición
Desconocido	@	diagnóstico léxico

Trabajo con la guía práctica

Guía + scripts

Paso 7

- ▶ Ejecutar y explicar el error de `error.imp`.
- ▶ Habilitar la última prueba.
- ▶ Agregar casos propios y preparar evidencias.

Qué construimos y qué automatiza LARK

Construimos

- ▶ categorías y nombres;
- ▶ patrones y prioridades;
- ▶ atributos y tabla;
- ▶ pruebas y diagnósticos.

LARK apoya

- ▶ aplicación de reglas;
- ▶ recorrido de la entrada;
- ▶ creación de tokens;
- ▶ seguimiento de posiciones.

La herramienta ejecuta una especificación; no decide el lenguaje por nosotros.

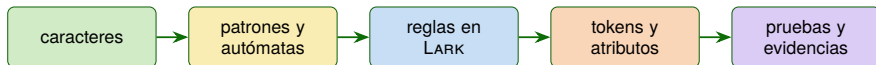
Antes de entregar

- ▶ ☐ No quedan TODO de la práctica.
- ▶ ☐ Todas las pruebas están habilitadas y pasan.
- ▶ ☐ `requirements.txt` fija la versión de LARK.
- ▶ ☐ El `README.md` permite repetir instalación y ejecución.
- ▶ ☐ El reporte contiene decisiones, evidencias, errores y tiempos.
- ▶ ☐ El historial muestra incrementos verificables.

Contrato para continuar

La lista de nombres de token y sus atributos debe permanecer estable al integrar el parser.

Conclusión



Idea de cierre

El lexer materializa el primer contrato interno del compilador. Su corrección depende de especificaciones explícitas, decisiones documentadas y pruebas que hagan visible cada transformación.

Referencias

1. Winskel, G. (1993). *The Formal Semantics of Programming Languages: An Introduction*. The MIT Press.
2. Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
3. Thain, D. (2026). *Introduction to Compilers and Language Design* (2.^a ed.). University of Notre Dame.
4. Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.
5. Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.
6. Lark. (s.f.). *API Reference y Grammar Reference*. <https://lark-parser.readthedocs.io/>